

Chapter 1. 디자인 패턴과

프로그래밍 패러다임.

Section 1. 디자인 패턴.

① 싱글톤 패턴.

- 하나의 클래스에 오직 하나의 인스턴스만 가지는 패턴.

- 장점 : 하나의 인스턴스로 다른 모듈들이 공유
하므로 인스턴스 생산비용 감소.

- 단점 : 의존성 ↑,
TDD에서의 경미한 테스트마다 컴파일 X

- 의존성 높은 단점을 해결하기 위한 DI
의존성 주입자를 통해 결합을 낮춘다.

- 단점 : 클래스 수 ↑, 약간의 런타임 패리티!

② 팩토리 패턴

- 객체 생성부분을 떼어내 추상화한 패턴.

(상위클래스 → 인터페이스)

(하위클래스 → 구체적인 내용.)

→ 상위·하위가 분리되어 느슨한 결합.

③ 전략 패턴 (정책 패턴)

- 객체의 행위를 바꿀수있는 경우 '캡슐화'한 행위를 컨텍스트 안에서 변경.

ex) 소셜로그인 (구글, 카카오 ...)

④ 옵저버 패턴

- 특정 객체가 상태 변화를 관찰하다가 메시지 등을 통해 변화를 알려줌.

ex) SNS 주제 팔로우, MVC 패턴.

- 프록시 객체를 통해 구현도 가능.

⑤ 프록시 패턴과 프록시 서버

- 대상 객체에 접근하기 전 그 접근에 대한 흐름을 가로채 로직을 수행하는 계층.
- 프록시 서버 : 클라이언트와 서버 사이
클라이언트가 프록시 서버를 통해 네트워크 서비스에
간접적으로 접속 가능하도록 해주는 컴퓨터 시스템
이나 응용프로그램.
ex) nginx, CloudFlare

⑥ 이터레이터 패턴.

- iterator를 사용하여 컬렉션의 요소들에 접근.
- ex) for A of B
(B는 list, set, map 가능.)

⑦ 노출모듈 패턴.

- private, public 같은 접근 제어를 만드는 패턴.

⑧ MVC 패턴.

- 애플리케이션을 Model, View, Controller로 구분.
- 장점 : 재사용성 확장성 용이
단점 : 애플리케이션이 복잡해질수록 모델 뷰 관계복잡

ex) Spring.

⑨ MVP 패턴.

- MVC 패턴 파생. Controller → Presenter로 교체.

뷰와 프레젠테이션은 1:1 관계. → 더 강한 결합.

⑩ MVVM 패턴.

- MVC 패턴 파생. Controller → View Model로 교체.

커맨드와 데이터 바인딩을 가지는 것이 특징.

뷰와 뷰모델 사이 양방향 데이터 바인딩 지원

• 장점: UI를 코드 수정없이 재사용 가능. 단위 테스트 쉬움.

ex) Vue.js